

### 03 · REST autenticado con API Key

Los notebooks 01 y 02 ya usaron el header `x-api-key` . Ahora la **autenticación es el tema**: qué pasa sin credencial o con una inválida, qué son los *scopes* y cómo se gestiona una clave.

La key demo `sk_demo_0000...` se re-siembra en cada arranque ( `tienda-virtual/.env.example` → `DEMO_API_KEY` ). Se generan y revocan más en el panel `/admin/api-keys` (requiere login; eso se ve en el notebook 07).

```
import csv
import os

import requests

BASE = "http://localhost:3000"
API_KEY = "sk_demo_00000000000000000000000000000000"
TIMEOUT = 30
```

#### Sin credencial: 401 unauthorized

El servidor rechaza la petición y explica en el cuerpo qué header espera.

```
r = requests.get(f"{BASE}/api/productos", timeout=TIMEOUT)
print("Sin API key ->", r.status_code, r.json())
```

Sin API key -> 401 {'error': 'unauthorized', 'message': "Envia el header 'x-api-key: <key>' o 'Authorization: Bearer <token OAuth>'."}

#### Con una key inválida: 401 invalid\_api\_key

Un `error` distinto: el servidor recibió una credencial pero no la reconoce (o fue revocada).

```
r = requests.get(
    f"{BASE}/api/productos",
    headers={"x-api-key": "sk_live_clave_que_no_existe_000000000000"},
    timeout=TIMEOUT,
)
print("Con key inválida ->", r.status_code, r.json())
```

Con key inválida -> 401 {'error': 'invalid\_api\_key', 'message': 'La API key no es valida o fue revocada.'}

#### Con la key demo válida

La cargamos en la `Session` y la hereda cada request.

```

sesion = requests.Session()
sesion.headers.update({"x-api-key": API_KEY, "User-Agent": "MineriaWeb-2026-2/1.0"})

r = sesion.get(f"{BASE}/api/testimonios", params={"page": 1, "pageSize": 3},
timeout=TIMEOUT)
r.raise_for_status()
print("OK ->", r.status_code, "| pageInfo:", r.json()["pageInfo"])

```

OK -> 200 | pageInfo: {'page': 1, 'pageSize': 3, 'total': 135, 'totalPages': 45}

## Scopes: read vs write

Cada credencial lleva scopes. Los GET necesitan read ; las mutaciones ( POST / PUT / DELETE ) necesitan write . La key demo tiene **ambos**, así que puede crear y borrar. Una credencial de solo lectura recibiría 403 insufficient\_scope en un POST . Para scraping basta read .

(La base de datos es SQLite en memoria y se re-siembr en cada arranque, así que este alta/baja de prueba no deja rastro.)

```

nuevo = sesion.post(
    f"{BASE}/api/productos",
    json={"codigo": "TMP-03", "nombre": "Producto de prueba (scope write)",
        "descripcion": "alta temporal", "precio": 1.0, "stock": 1},
    timeout=TIMEOUT,
)
print("POST ->", nuevo.status_code, "(201 = creado; el scope 'write' lo permite)")
creado_id = nuevo.json()["id"]

borrado = sesion.delete(f"{BASE}/api/productos/{creado_id}", timeout=TIMEOUT)
print("DELETE ->", borrado.status_code, borrado.json())

```

POST -> 201 (201 = creado; el scope 'write' lo permite)  
DELETE -> 200 {'eliminado': True, 'id': 91}

## Scraping con la key: todos los testimonios

Recorremos el listado paginado completo de /api/testimonios (comentarios generales de clientes).

```

def descargar_todo(path, page_size=50, **filtros):
    items, page = [], 1
    while True:
        payload = sesion.get(
            f"{BASE}/{path}", params={**filtros, "page": page, "pageSize": page_size},
            timeout=TIMEOUT
        ).json()
        items.extend(payload["items"])
        info = payload["pageInfo"]
        print(f"  página {info['page']}>2}/{info['totalPages']} (total
{len(items)}/{info['total']})")
        if info["page"] >= info["totalPages"]:
            return items
        page += 1

```

```
testimonios = descargar_todo("/api/testimonios")
print(f"\n{len(testimonios)} testimonios")
```

```
página 1/3 (total 50/135)
página 2/3 (total 100/135)
página 3/3 (total 135/135)
```

135 testimonios

## Guardar en data/rest\_testimonios.csv

```
DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
OUTPUT = os.path.join(DATA_DIR, "rest_testimonios.csv")

with open(OUTPUT, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(
        f,
        fieldnames=["id", "cliente_id", "cliente_nombre", "cliente_ciudad",
"cliente_pais", "calificacion", "fecha", "texto"],
    )
    writer.writeheader()
    for t in testimonios:
        writer.writerow({
            "id": t["id"],
            "cliente_id": t["clienteId"],
            "cliente_nombre": f"{t['clienteNombre']} {t['clienteApellidos']}",
            "cliente_ciudad": t["clienteCiudad"],
            "cliente_pais": t["clientePais"],
            "calificacion": t["calificacion"],
            "fecha": t["fecha"],
            "texto": t["texto"],
        })

print(f"Guardadas {len(testimonios)} filas en {OUTPUT}")
```

Guardadas 135 filas en /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/rest\_testimonios.csv

La API key es un secreto de **larga vida**: si se filtra, hay que revocarla a mano en el panel. Para integraciones máquina-a-máquina se prefiere OAuth2, con tokens de vida corta que caducan solos. Eso es el siguiente notebook.